

LAPORAN PROJECT EAS
PEMROGRAMAN BERORIENTASI OBJEK (14823274)



KELAS : B

NAMA KELOMPOK :

MUH ZAKY SAIFULLOH Y (1482500057)

RAFI ARDIAN SAPUTRA (1482500046)

ZAKY AKSA KAFA BILLAH (1482500059)

Dosen Pengampu:

Bagus Hardiansyah, S.Kom., M.Si.

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
JUNI 2026

DAFTAR ISI

COVER	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1Latar Belakang	1
1.2Rumusan Masalah.....	2
1.3Tujuan Penelitian	2
1.4Manfaat Penelitian	3
BAB II LANDASAN TEORI	4
2.1Pengertian Python	4
2.2Object Oriented Programming (oop).....	4
2.3Class dan Object.....	5
2.3.1 Class.....	5
2.3.2 Object	6
2.4Encapsulation	6
2.5Polymorphism.....	7
2.6Abstraction.....	7
2.7Hubungan Encapsulation dan Polymorphism.....	8
BAB III ANALISIS DAN PERANCANGAN SISTEM.....	9
3.1Analisis Sistem	9
3.2Perancangan Alur Sistem(Flowchart)	9
3.3Perancangan Class Diagram	10
3.4Gambaran Cara Kerja Sistem	11
BAB IV IMPLEMENTASI DAN PEMBAHASAN	12
4.1Implementasi Program	12
4.2Implementasi Class Tiket Bioskop.....	12
4.3Implementasi Getter dan Setter	13
4.4Implementasi Abstract Class.....	13
4.5Implementasi Class Tiket Regular	14
4.6Implementasi Class Tiket VIP	14

4.7Implementasi Konsep Encapsulation	14
4.8Hasil Pengujian Program	15
BAB V PENUTUP.....	16
5.1Kesimpulan	16
5.2Saran.....	16

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi telah memberikan dampak yang signifikan terhadap berbagai aspek kehidupan, salah satunya pada bidang hiburan. Saat ini hampir seluruh aktivitas yang sebelumnya dilakukan secara manual telah beralih ke sistem berbasis komputer, termasuk dalam proses pemesanan tiket bioskop. Sistem pemesanan tiket secara digital memberikan kemudahan bagi pelanggan dalam memilih film, menentukan jadwal tayang, memilih kursi, serta melakukan transaksi dengan lebih cepat dan efisien. Selain meningkatkan kualitas pelayanan, penggunaan sistem komputerisasi juga membantu pihak pengelola bioskop dalam mengurangi kesalahan pencatatan data dan meningkatkan efektivitas pengelolaan informasi.

Dalam dunia pendidikan, khususnya pada mata kuliah Pemrograman Berorientasi Objek (PBO), mahasiswa dituntut untuk tidak hanya memahami teori, tetapi juga mampu mengimplementasikan konsep-konsep Object-Oriented Programming (OOP) ke dalam sebuah aplikasi nyata. Salah satu studi kasus yang relevan adalah pembuatan Sistem Pemesanan Tiket Bioskop. Studi kasus ini dipilih karena memiliki alur kerja yang mudah dipahami serta melibatkan berbagai objek yang saling berinteraksi, seperti tiket, film, pelanggan, dan jenis layanan. Dengan demikian, mahasiswa dapat lebih mudah memahami penerapan konsep OOP dalam menyelesaikan permasalahan yang sering dijumpai dalam kehidupan sehari-hari.

Bahasa pemrograman Python dipilih sebagai media pengembangan aplikasi karena memiliki sintaks yang sederhana, mudah dipelajari, serta mendukung implementasi paradigma pemrograman berorientasi objek secara lengkap. Python juga menyediakan berbagai fitur yang mempermudah proses pengembangan aplikasi, seperti penggunaan class, object, inheritance, encapsulation, polymorphism, abstraction, serta berbagai pustaka pendukung lainnya. Oleh karena itu, Python menjadi salah satu bahasa pemrograman yang banyak digunakan dalam proses pembelajaran maupun pengembangan perangkat lunak.

Pada proyek ini, konsep **Encapsulation** diterapkan untuk menjaga keamanan data yang dimiliki setiap objek. Data penting seperti identitas tiket, judul film, nomor kursi, dan harga dasar disimpan sebagai atribut private sehingga tidak dapat diakses secara langsung dari luar class. Akses terhadap data tersebut dilakukan melalui method getter dan setter yang berfungsi untuk mengontrol proses pengambilan maupun perubahan data. Dengan adanya encapsulation, integritas data dapat terjaga dan risiko kesalahan akibat perubahan data secara langsung dapat diminimalkan.

Selain encapsulation, proyek ini juga menerapkan konsep **Polymorphism**. Konsep ini memungkinkan suatu method memiliki perilaku yang berbeda meskipun memiliki nama yang sama. Dalam sistem pemesanan tiket bioskop, setiap jenis tiket, seperti tiket reguler, tiket VIP,

maupun tiket premiere, memiliki cara perhitungan harga yang berbeda. Melalui polymorphism, setiap class turunan dapat mengimplementasikan method `hitung_harga()` sesuai aturan masing-masing tanpa mengubah struktur program utama. Pendekatan ini membuat program menjadi lebih fleksibel, mudah dikembangkan, dan mempermudah proses penambahan jenis tiket baru di masa mendatang.

Berdasarkan uraian tersebut, penulis mengembangkan sebuah aplikasi **Sistem Pemesanan Tiket Bioskop dengan Konsep Encapsulation dan Polymorphism Menggunakan Python**. Diharapkan proyek ini dapat menjadi media pembelajaran dalam memahami implementasi konsep Pemrograman Berorientasi Objek sekaligus memberikan gambaran mengenai penerapan OOP pada sistem informasi sederhana yang sering dijumpai dalam kehidupan sehari-hari.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah pada proyek ini adalah sebagai berikut.

1. Bagaimana merancang dan membangun Sistem Pemesanan Tiket Bioskop menggunakan bahasa pemrograman Python dengan pendekatan Object-Oriented Programming (OOP)?
2. Bagaimana mengimplementasikan konsep Encapsulation untuk menjaga keamanan dan validitas data pada sistem pemesanan tiket bioskop?
3. Bagaimana mengimplementasikan konsep Polymorphism agar setiap jenis tiket memiliki mekanisme perhitungan harga yang berbeda sesuai karakteristiknya?
4. Bagaimana menghasilkan aplikasi yang mudah digunakan, mudah dikembangkan, dan mampu memberikan informasi pemesanan tiket secara jelas kepada pengguna?

1.3 Tujuan Penelitian

Tujuan yang ingin dicapai dalam pembuatan proyek ini adalah sebagai berikut.

1. Merancang dan membangun aplikasi Sistem Pemesanan Tiket Bioskop menggunakan bahasa pemrograman Python.
2. Mengimplementasikan konsep Pemrograman Berorientasi Objek (Object-Oriented Programming) pada proses pengembangan aplikasi.

3. Menerapkan konsep Encapsulation untuk melindungi data objek melalui penggunaan atribut private serta method getter dan setter.
4. Menerapkan konsep Polymorphism dalam proses perhitungan harga tiket berdasarkan jenis tiket yang dipilih.
5. Meningkatkan pemahaman mengenai penerapan konsep OOP melalui implementasi pada studi kasus nyata.

1.4 Manfaat Penelitian

Adapun manfaat yang diperoleh dari pembuatan proyek ini adalah sebagai berikut.

1. Bagi Mahasiswa

Proyek ini dapat meningkatkan pemahaman mahasiswa mengenai konsep-konsep dasar Pemrograman Berorientasi Objek, khususnya Encapsulation dan Polymorphism. Selain itu, mahasiswa memperoleh pengalaman dalam mengembangkan aplikasi sederhana menggunakan bahasa pemrograman Python.

2. Bagi Proses Pembelajaran

Aplikasi yang dibuat dapat dijadikan sebagai contoh implementasi OOP pada mata kuliah Pemrograman Berorientasi Objek sehingga dapat membantu mahasiswa lain dalam memahami materi yang telah dipelajari selama perkuliahan.

3. Bagi Pengembangan Sistem

Sistem ini dapat menjadi dasar dalam pengembangan aplikasi yang lebih kompleks, seperti penambahan database, sistem login pengguna, integrasi pembayaran digital, pemilihan jadwal film, hingga pengembangan antarmuka berbasis web maupun desktop.

BAB II

LANDASAN TEORI

2.1 Pengertian Python

Python merupakan salah satu bahasa pemrograman tingkat tinggi (*high-level programming language*) yang dikembangkan oleh **Guido van Rossum** dan pertama kali dirilis pada tahun 1991. Python dirancang dengan sintaks yang sederhana sehingga mudah dipelajari oleh pemula, namun tetap memiliki kemampuan yang sangat kuat untuk membangun berbagai jenis aplikasi, mulai dari aplikasi desktop, web, data science, kecerdasan buatan (*Artificial Intelligence*), hingga sistem berbasis jaringan.

Python menggunakan paradigma pemrograman yang beragam (*multi-paradigm*), seperti pemrograman prosedural, fungsional, dan terutama **Pemrograman Berorientasi Objek (Object-Oriented Programming/OOP)**. Kemampuan tersebut menjadikan Python sebagai salah satu bahasa pemrograman yang paling banyak digunakan dalam dunia pendidikan maupun industri.

Pada proyek **Sistem Pemesanan Tiket Bioskop**, Python digunakan sebagai bahasa utama karena memiliki sintaks yang sederhana sehingga memudahkan proses implementasi konsep OOP, khususnya Encapsulation dan Polymorphism. Selain itu, Python juga menyediakan berbagai modul bawaan yang membantu proses pengembangan aplikasi menjadi lebih efisien.

Adapun beberapa kelebihan Python adalah sebagai berikut.

1. Memiliki sintaks yang sederhana dan mudah dipahami.
2. Mendukung konsep Object-Oriented Programming secara lengkap.
3. Bersifat open source sehingga dapat digunakan secara gratis.
4. Memiliki komunitas yang besar serta dokumentasi yang lengkap.
5. Dapat dijalankan pada berbagai sistem operasi seperti Windows, Linux, dan macOS.

2.2 Object Oriented Programming (oop)

Object-Oriented Programming (OOP) atau Pemrograman Berorientasi Objek merupakan paradigma pemrograman yang berfokus pada penggunaan **objek (object)** sebagai dasar dalam membangun sebuah

program. Setiap objek memiliki data yang disebut **atribut** (*attribute*) dan perilaku yang disebut **method**.

Konsep OOP bertujuan untuk membuat program menjadi lebih terstruktur, mudah dipelihara (*maintainable*), mudah dikembangkan (*scalable*), serta mempermudah penggunaan kembali kode (*reusability*). Dengan menggunakan pendekatan OOP, suatu sistem dapat dibagi menjadi beberapa objek yang saling berinteraksi sesuai dengan fungsinya masing-masing.

Dalam proyek Sistem Pemesanan Tiket Bioskop, setiap tiket direpresentasikan sebagai sebuah objek yang memiliki atribut seperti ID tiket, judul film, nomor kursi, dan harga. Selain itu, objek juga memiliki method untuk menghitung harga tiket dan menampilkan informasi pemesanan.

Secara umum, OOP memiliki empat konsep utama, yaitu:

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

Pada proyek ini, konsep yang difokuskan adalah Encapsulation dan Polymorphism.

2.3 Class dan Object

2.3.1 Class

Object merupakan hasil implementasi dari sebuah class. Apabila class diibaratkan sebagai rancangan sebuah rumah, maka object adalah rumah yang telah dibangun berdasarkan rancangan tersebut.

Dalam proyek ini, ketika pengguna memesan tiket bioskop, maka sistem akan membuat sebuah objek tiket yang berisi informasi sesuai dengan data yang dimasukkan pengguna.

Contoh pembuatan object adalah sebagai berikut.

```
class TiketBioskop  
  
pass
```

Class tersebut belum memiliki atribut maupun method, tetapi sudah dapat digunakan sebagai dasar pembuatan objek.

2.3.2 Object

Dalam proyek ini, ketika pengguna memesan tiket bioskop, maka sistem akan membuat sebuah objek tiket yang berisi informasi sesuai dengan data yang dimasukkan pengguna.

Contoh pembuatan object adalah sebagai berikut.

```
tiket = TiketBioskop()
```

Object tersebut nantinya dapat digunakan untuk memanggil atribut maupun method yang terdapat pada class.

2.4 Encapsulation

Encapsulation merupakan salah satu konsep penting dalam Object-Oriented Programming yang berfungsi untuk menyembunyikan data (*data hiding*) agar tidak dapat diakses secara langsung dari luar class. Data hanya dapat diakses melalui method tertentu, seperti getter dan setter.

Tujuan utama Encapsulation adalah menjaga keamanan data serta mencegah terjadinya perubahan nilai atribut secara sembarangan yang dapat menyebabkan kesalahan pada program.

Dalam bahasa Python, Encapsulation dapat diterapkan dengan menggunakan atribut private yang diawali dengan dua garis bawah (`__`)

Sebagai contoh pada proyek ini terdapat atribut berikut.

```
self.__id_tiket  
self.__judul_film  
self.__kursi  
self.__harga_dasar
```

Atribut tersebut tidak dapat diakses secara langsung dari luar class. Untuk mengambil maupun mengubah nilainya digunakan method getter dan setter.

Contoh getter:

```
@property  
def harga(self):  
return self.__harga_dasar
```

Contoh Setter :

```
@harga.setter
```

```
def harga(self, nilai):  
    if nilai > 0:  
        self.__harga_dasar = nilai
```

Penerapan Encapsulation membuat data pada sistem pemesanan tiket menjadi lebih aman karena setiap perubahan nilai harus melalui proses validasi terlebih dahulu.

2.5 Polymorphism

Polymorphism berasal dari kata *poly* yang berarti banyak dan *morph* yang berarti bentuk. Dalam OOP, Polymorphism merupakan kemampuan suatu method untuk memiliki perilaku yang berbeda meskipun memiliki nama yang sama.

Pada proyek Sistem Pemesanan Tiket Bioskop, setiap jenis tiket memiliki method `hitung_harga()`, namun cara perhitungan harga berbeda sesuai dengan jenis tiket yang dipilih.

Sebagai contoh:

- Tiket Reguler menggunakan harga dasar.
- Tiket VIP menambahkan biaya layanan VIP.
- Tiket Premiere menambahkan biaya premium.

Meskipun method yang digunakan sama, hasil yang diperoleh akan berbeda karena masing-masing class mengimplementasikan logika perhitungan yang berbeda.

Keuntungan penggunaan Polymorphism adalah program menjadi lebih fleksibel serta mempermudah proses pengembangan apabila di kemudian hari ditambahkan jenis tiket baru tanpa perlu mengubah kode utama.

2.6 Abstraction

Abstraction merupakan konsep dalam OOP yang digunakan untuk menyembunyikan detail implementasi dan hanya menampilkan fungsi-fungsi penting kepada pengguna.

Dalam Python, Abstraction dapat diterapkan menggunakan modul **ABC (Abstract Base Class)**. Class abstrak tidak dapat dibuat menjadi objek secara langsung, tetapi digunakan sebagai dasar bagi class turunannya.

Pada proyek ini, class **TiketBioskop** dijadikan sebagai class abstrak yang memiliki method **hitung_harga()** sebagai abstract method. Method tersebut wajib diimplementasikan oleh seluruh class turunan, seperti Tiket Reguler, Tiket VIP, dan Tiket Premiere.

Dengan adanya abstract class, struktur program menjadi lebih rapi serta memastikan bahwa setiap class turunan memiliki method yang sama sesuai kebutuhan sistem.

2.7 Hubungan Encapsulation dan Polymorphism

Pada aplikasi Sistem Pemesanan Tiket Bioskop, kedua konsep OOP tersebut saling melengkapi dalam membangun sistem yang lebih terstruktur. Konsep **Encapsulation** digunakan untuk melindungi data penting, seperti ID tiket, judul film, nomor kursi, dan harga dasar, sehingga tidak dapat diakses atau diubah secara langsung dari luar class. Proses pengambilan maupun perubahan data dilakukan melalui method getter dan setter yang telah disediakan.

Sementara itu, konsep **Polymorphism** digunakan pada proses perhitungan harga tiket. Setiap jenis tiket memiliki implementasi method `hitung_harga()` yang berbeda sesuai dengan karakteristiknya. Dengan demikian, program dapat menghitung harga tiket secara otomatis berdasarkan jenis tiket yang dipilih tanpa mengubah struktur program utama.

Penerapan kedua konsep tersebut menjadikan aplikasi lebih aman, mudah dipelihara, serta mudah dikembangkan apabila di masa mendatang ditambahkan fitur baru, seperti jenis tiket tambahan, sistem diskon, atau integrasi dengan basis data.

BAB III

ANALISIS DAN PERANCANGAN SISTEM

3.1 Analisis Sistem

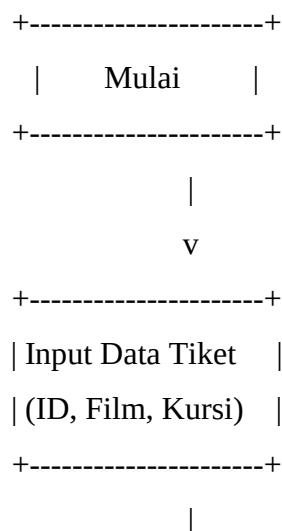
Analisis sistem merupakan tahapan awal dalam pengembangan perangkat lunak yang bertujuan untuk mengetahui kebutuhan sistem sebelum proses implementasi dilakukan. Pada proyek ini, sistem yang dikembangkan adalah **Sistem Pemesanan Tiket Bioskop** berbasis Python dengan menerapkan konsep Object-Oriented Programming (OOP), khususnya **Encapsulation** dan **Polymorphism**.

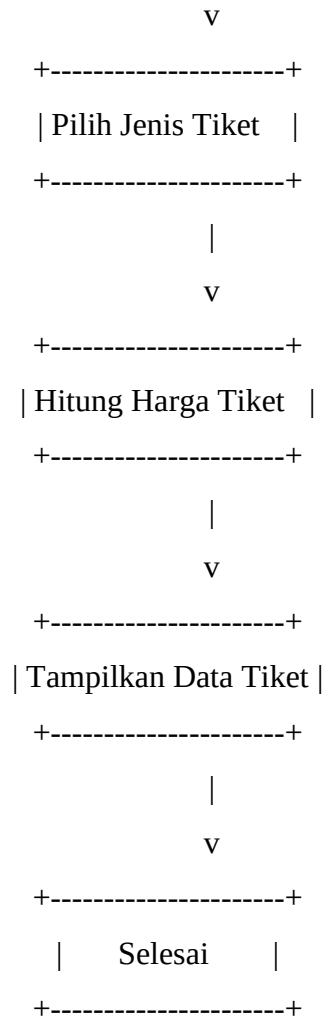
Sistem ini dirancang untuk membantu pengguna melakukan proses pemesanan tiket bioskop secara sederhana melalui terminal (console). Pengguna dapat memasukkan data pemesanan seperti ID tiket, judul film, nomor kursi, dan memilih jenis tiket. Setelah data dimasukkan, sistem akan menghitung harga tiket sesuai dengan jenis tiket yang dipilih, kemudian menampilkan informasi pemesanan secara lengkap.

Dengan adanya sistem ini, proses pemesanan tiket menjadi lebih terstruktur dibandingkan pencatatan secara manual. Selain itu, penggunaan konsep OOP membuat kode program lebih mudah dipahami, dipelihara, dan dikembangkan apabila di kemudian hari diperlukan penambahan fitur baru.

3.2 Perancangan Alur Sistem(Flowchart)

Flowchart digunakan untuk menggambarkan urutan proses yang terjadi pada sistem. Secara umum alur kerja aplikasi dapat dijelaskan sebagai berikut:



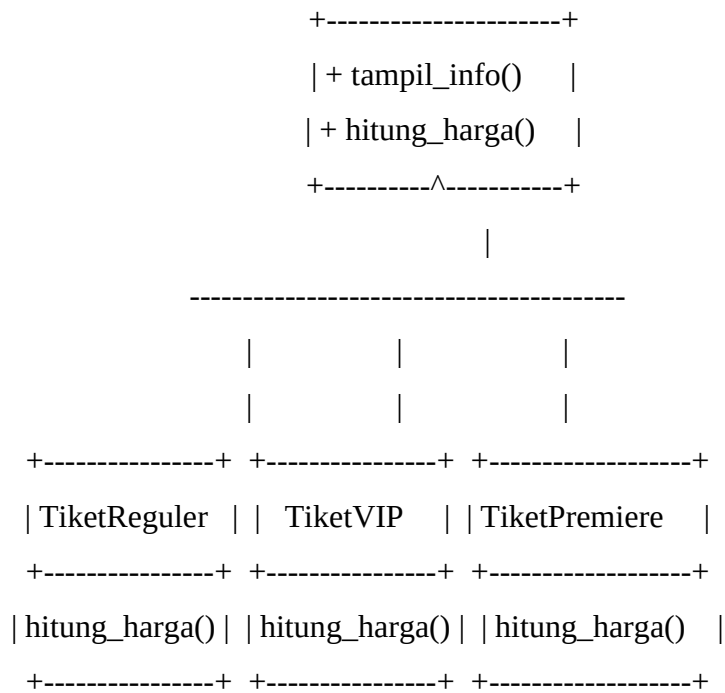


Flowchart tersebut menunjukkan bahwa proses dimulai dari input data, dilanjutkan dengan pemilihan jenis tiket, perhitungan harga, hingga sistem menampilkan hasil pemesanan kepada pengguna.

3.3 Perancangan Class Diagram

Pada proyek ini digunakan pendekatan Object-Oriented Programming sehingga sistem dibangun menggunakan beberapa class yang saling berhubungan. Struktur class secara sederhana dapat digambarkan sebagai berikut.





Pada diagram tersebut, **TiketBioskop** berperan sebagai *parent class* atau class dasar yang menyimpan atribut umum. Selanjutnya terdapat beberapa class turunan yang memiliki implementasi method `hitung_harga()` masing-masing. Hal ini menunjukkan penerapan konsep **Polymorphism**, karena setiap class memiliki cara perhitungan harga yang berbeda.

3.4 Gambaran Cara Kerja Sistem

Secara keseluruhan, aplikasi dimulai ketika pengguna menjalankan program melalui terminal. Pengguna kemudian diminta memasukkan informasi tiket yang terdiri atas ID tiket, judul film, nomor kursi, dan jenis tiket. Setelah seluruh data diterima, sistem membuat objek berdasarkan jenis tiket yang dipilih.

Objek tersebut akan menjalankan method `hitung_harga()` sesuai implementasi pada class masing-masing. Setelah harga berhasil dihitung, sistem menampilkan seluruh informasi pemesanan kepada pengguna sebagai hasil akhir proses.

Alur tersebut menunjukkan bahwa konsep **Encapsulation** digunakan untuk mengelola data objek secara aman, sedangkan **Polymorphism** digunakan untuk memberikan fleksibilitas dalam perhitungan harga tiket berdasarkan jenis tiket yang dipilih.

BAB IV

IMPLEMENTASI DAN PEMBAHASAN

4.1 Implementasi Program

Tahap implementasi merupakan proses penerapan hasil perancangan sistem ke dalam bentuk program menggunakan bahasa pemrograman Python. Program ini dikembangkan dengan menerapkan konsep **Object-Oriented Programming (OOP)** sehingga struktur kode menjadi lebih rapi, mudah dipahami, dan mudah dikembangkan.

Pada aplikasi ini terdapat beberapa class yang saling berhubungan. Setiap class memiliki tugas masing-masing dalam menjalankan fungsi sistem pemesanan tiket bioskop. Selain itu, program juga menerapkan konsep **Encapsulation** untuk melindungi data serta **Polymorphism** untuk membedakan proses perhitungan harga berdasarkan jenis tiket.

Program dijalankan melalui terminal (console) sehingga pengguna dapat langsung melakukan proses pemesanan tiket dengan memasukkan data yang diminta oleh sistem.

4.2 Implementasi Class Tiket Bioskop

Class TiketBioskop merupakan class utama yang dijadikan sebagai dasar bagi seluruh jenis tiket. Class ini berfungsi untuk menyimpan atribut umum yang dimiliki setiap tiket serta menyediakan method yang nantinya digunakan oleh class turunan.

Beberapa atribut yang dimiliki class ini antara lain:

- ID Tiket
- Judul Film
- Nomor Kursi
- Harga Dasar

Seluruh atribut tersebut dibuat private menggunakan tanda `__` sehingga tidak dapat diakses secara langsung dari luar class.

Contoh implementasi:

```
class TiketBioskop(ABC):  
    def __init__(self,id_tiket,judul_film,kursi,harga):  
        self.__id_tiket=id_tiket  
        self.__judul_film=judul_film
```

```
self.__kursi=kursi
```

```
self.__harga=harga
```

Pada constructor di atas terlihat bahwa seluruh data disimpan sebagai atribut private. Hal ini merupakan implementasi konsep **Encapsulation** yang bertujuan menjaga keamanan data.

4.3 Implementasi Getter dan Setter

Agar atribut private tetap dapat digunakan oleh class lain, program menyediakan **Getter** dan **Setter**.

Getter digunakan untuk mengambil nilai atribut.

Contoh:

```
@property
```

```
def harga(self):
```

```
    return self.__harga
```

Method tersebut memungkinkan pengguna memperoleh nilai harga tanpa mengakses atribut private secara langsung.

Sedangkan Setter digunakan untuk mengubah nilai atribut sekaligus melakukan validasi terhadap data yang dimasukkan.

Contoh:

```
@harga.setter
```

```
def harga(self,nilai):
```

```
    if nilai > 0:
```

```
        self.__harga = nilai
```

Pada kode tersebut, harga hanya dapat diubah apabila nilainya lebih besar dari nol. Apabila pengguna memasukkan nilai negatif, maka perubahan tidak akan dilakukan. Penerapan setter seperti ini membantu menjaga validitas data sehingga sistem menjadi lebih aman.

4.4 Implementasi Abstract Class

Program memanfaatkan **Abstract Base Class (ABC)** sebagai dasar bagi seluruh jenis tiket.

Class abstrak memiliki method yang belum memiliki isi sehingga wajib diimplementasikan oleh class turunannya.

Contoh:

```
@abstractmethod  
def hitung_harga(self):  
pass
```

Method `hitung_harga()` hanya berfungsi sebagai template. Seluruh class turunan diwajibkan membuat implementasi method tersebut sesuai kebutuhan masing-masing.

Penggunaan abstract class membuat struktur program menjadi lebih konsisten karena setiap jenis tiket memiliki method yang sama.

4.5 Implementasi Class Tiket Regular

Class **TiketReguler** merupakan turunan dari class **TiketBioskop**.

Class ini digunakan untuk menghitung harga tiket reguler.

Contoh implementasi:

```
class TiketReguler(TiketBioskop):  
def hitung_harga(self):  
return self.harga
```

Method tersebut mengembalikan harga dasar tanpa adanya tambahan biaya sehingga harga tiket reguler sama dengan harga awal yang telah ditentukan.

4.6 Implementasi Class Tiket VIP

Class **TiketVIP** juga merupakan turunan dari class **TiketBioskop**.

Perbedaannya terletak pada proses perhitungan harga.

Contoh:

```
class TiketVIP(TiketBioskop):  
def hitung_harga(self):  
return self.harga + 25000
```

Harga tiket VIP diperoleh dari harga dasar yang ditambah biaya layanan VIP sebesar Rp25.000.

Implementasi ini menunjukkan bahwa method yang sama dapat menghasilkan nilai berbeda.

4.7 Implementasi Konsep Encapsulation

Konsep Encapsulation diterapkan dengan menjadikan seluruh atribut utama sebagai atribut private.

Contohnya:

self.__id_tiket

self.__judul_film

self.__kursi

self.__harga

Dengan penggunaan atribut private, data tidak dapat diakses secara langsung dari luar class. Seluruh proses pengambilan maupun perubahan data harus melalui getter dan setter.

Penerapan Encapsulation memberikan beberapa keuntungan, yaitu:

- Melindungi data dari perubahan secara langsung.
- Menjaga konsistensi nilai atribut.
- Memudahkan proses validasi data.
- Membuat program lebih aman.

4.8 Hasil Pengujian Program

Setelah seluruh class selesai diimplementasikan, program dijalankan menggunakan terminal Python.

Pengguna diminta memasukkan data berupa:

- ID Tiket
- Judul Film
- Nomor Kursi
- Jenis Tiket

Setelah seluruh data dimasukkan, sistem akan menghitung harga sesuai jenis tiket dan menampilkan informasi pemesanan.

Contoh hasil keluaran program:

```
Berhasil menambahkan pesanan: T-001
Berhasil menambahkan pesanan: T-002
Berhasil menambahkan pesanan: T-003

=== STRUK PEMESANAN TIKET BIOSKOP ===
[REGULER] Dune: Part Two | Kursi: A12 | Total: Rp50,000.00
[VIP] Interstellar | Kursi: V01 | Akses Lounge: Ya | Total: Rp112,500.00
[IMAX 3D] Oppenheimer | Kursi: M15 | Kacamata 3D Included | Total: Rp103,000.00
-----
Total Pembayaran: Rp265,500.00
```

Hasil pengujian menunjukkan bahwa program berhasil menjalankan seluruh fungsi yang telah dirancang. Data tiket dapat disimpan dengan baik melalui konsep **Encapsulation**, sedangkan proses perhitungan harga dapat dilakukan secara otomatis menggunakan konsep **Polymorphism**. Dengan demikian, aplikasi telah memenuhi tujuan utama yaitu membangun sistem pemesanan tiket bioskop yang terstruktur, mudah dipahami, dan sesuai dengan prinsip-prinsip Pemrograman Berorientasi Objek.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil analisis, perancangan, implementasi, dan pengujian program yang telah dilakukan, dapat disimpulkan bahwa aplikasi **Sistem Pemesanan Tiket Bioskop** berhasil dikembangkan menggunakan bahasa pemrograman Python dengan menerapkan paradigma **Object-Oriented Programming (OOP)**. Program ini mampu melakukan proses pemesanan tiket mulai dari memasukkan data pengguna, memilih jenis tiket, menghitung harga tiket, hingga menampilkan informasi pemesanan secara lengkap melalui terminal.

Penerapan konsep **Encapsulation** pada program berhasil meningkatkan keamanan dan konsistensi data. Hal ini dibuktikan dengan penggunaan atribut bertipe private yang hanya dapat diakses melalui method getter dan setter. Dengan cara tersebut, setiap perubahan terhadap data penting seperti ID tiket, judul film, nomor kursi, dan harga dasar dapat dikontrol melalui proses validasi sehingga mengurangi risiko kesalahan akibat perubahan data secara langsung.

Selain itu, konsep **Polymorphism** juga telah diterapkan dengan baik melalui method `hitung_harga()`. Meskipun setiap class turunan menggunakan nama method yang sama, proses perhitungan harga dilakukan sesuai dengan karakteristik masing-masing jenis tiket. Pendekatan ini membuat struktur program menjadi lebih fleksibel, karena penambahan jenis tiket baru dapat dilakukan tanpa mengubah logika utama yang telah ada.

Penggunaan **Abstract Class** sebagai class dasar juga membantu menjaga konsistensi struktur program. Setiap class turunan diwajibkan mengimplementasikan method yang telah didefinisikan pada class induk sehingga proses pengembangan program menjadi lebih terarah dan terorganisasi.

Secara keseluruhan, aplikasi yang dibuat telah memenuhi tujuan utama proyek, yaitu mengimplementasikan konsep **Encapsulation** dan **Polymorphism** dalam sebuah studi kasus nyata berupa sistem pemesanan tiket bioskop. Selain berfungsi sebagai media pembelajaran, aplikasi ini juga memberikan gambaran mengenai bagaimana konsep-konsep Object-Oriented Programming diterapkan dalam pengembangan perangkat lunak sederhana.

5.2 Saran

Walaupun aplikasi telah mampu menjalankan fungsi utama pemesanan tiket bioskop, masih terdapat beberapa pengembangan yang dapat dilakukan agar sistem menjadi lebih lengkap dan lebih mendekati aplikasi yang digunakan pada dunia industri.

Beberapa saran pengembangan yang dapat dilakukan antara lain sebagai berikut.

1. Menambahkan Database

Pada proyek ini seluruh data masih disimpan sementara selama program berjalan. Pengembangan selanjutnya dapat menggunakan database seperti MySQL atau SQLite agar data tiket, daftar film, jadwal tayang, dan riwayat pemesanan dapat disimpan secara permanen.

2. Membuat Antarmuka Grafis (GUI)

Program saat ini masih menggunakan tampilan berbasis terminal (*console*). Pengembangan berikutnya dapat memanfaatkan pustaka seperti Tkinter, PyQt, atau membuat aplikasi berbasis web menggunakan Flask maupun Django agar tampilan menjadi lebih menarik dan mudah digunakan oleh pengguna.

3. Menambahkan Sistem Login

Sistem dapat dikembangkan dengan menambahkan fitur login untuk membedakan hak akses antara administrator dan pelanggan. Administrator dapat mengelola data film, jadwal tayang, serta harga tiket, sedangkan pelanggan hanya dapat melakukan pemesanan.

4. Menambahkan Fitur Pemilihan Jadwal dan Studio

Saat ini sistem hanya berfokus pada proses pemesanan tiket. Ke depannya aplikasi dapat dikembangkan dengan menambahkan pilihan jadwal penayangan, nomor studio, kapasitas kursi, serta status kursi yang tersedia sehingga proses pemesanan menjadi lebih realistis.

5. Integrasi Metode Pembayaran

Sistem dapat dilengkapi dengan simulasi pembayaran menggunakan berbagai metode, seperti transfer bank, dompet digital (*e-wallet*), QRIS, maupun kartu debit. Dengan adanya fitur tersebut, aplikasi akan lebih menyerupai sistem pemesanan tiket bioskop yang sebenarnya.

6. Penerapan Konsep OOP yang Lebih Lengkap

Meskipun proyek ini telah menerapkan konsep Encapsulation, Polymorphism, dan Abstract Class, pengembangan selanjutnya dapat mengoptimalkan penggunaan konsep OOP lainnya, seperti Inheritance dan Composition, untuk memperluas struktur aplikasi dan meningkatkan modularitas kode.

Dengan adanya berbagai pengembangan tersebut, diharapkan aplikasi Sistem Pemesanan Tiket Bioskop tidak hanya menjadi media pembelajaran mengenai Object-Oriented Programming, tetapi juga dapat berkembang menjadi aplikasi yang lebih kompleks, efisien, dan sesuai dengan kebutuhan pengguna.

DAFTAR PUSTAKA

- Booch, G. (2007).** *Object-Oriented Analysis and Design with Applications* (3rd ed.). Addison-Wesley.
- Downey, A. (2015).** *Think Python: How to Think Like a Computer Scientist* (2nd ed.). O'Reilly Media.
- Fowler, M. (2004).** *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995).** *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Lutz, M. (2013).** *Learning Python* (5th ed.). O'Reilly Media.
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
- Python Software Foundation. (2025). *abc — Abstract Base Classes*.
- Diakses pada 29 Juni 2026, dari <https://docs.python.org/3/library/abc.html>**
- Python Software Foundation. (2025). *Built-in Functions*.
- Diakses pada 29 Juni 2026, dari <https://docs.python.org/3/library/functions.html>**
- Python Software Foundation. (2025). *Python Documentation*.
- Diakses pada 29 Juni 2026, dari <https://docs.python.org/3/>**
- Python Software Foundation. (2025). *Python Tutorial*. **Diakses pada 29 Juni 2026, dari <https://docs.python.org/3/tutorial/>**
- Schildt, H. (2018).** *Python: The Complete Reference*. McGraw-Hill Education.
- Sommerville, I. (2016).** *Software Engineering* (10th ed.). Pearson Education.
- Van Rossum, G., & Drake, F. L. (2009).** *Python 3 Reference Manual*. CreateSpace.